Vidyalankar Institute of Technology

Department of Computer Engineering

DEVOPS MINI PROJECT

Week 9 Report

Automated Grocery Inventory Dashboard

Name:	Prachi Patil
Roll No.:	23102B0038
Class:	BE - VII
Division:	B
Academic Year:	2026–27

1. Week 9 Objective

The objective of Week 9 was to introduce automated functional testing for the Automated Grocery Inventory Dashboard using Selenium WebDriver. The focus was on identifying critical user journeys and converting them into repeatable browser-based automated test cases.

The tests were implemented using Selenium WebDriver with Java and JUnit 5. The application was tested locally through the deployed Tomcat application, and Maven was used to execute the complete test suite.

The major objectives of Week 9 were:

- Identify three to five critical user journeys of the application.
- Design Selenium WebDriver test cases for the selected journeys.
- Add assertions to verify expected application behaviour.
- Use suitable and controlled test data.
- Implement automatic screenshots for failed Selenium tests.
- Execute the Selenium suite locally through Maven.
- Generate a local test report using Maven Surefire.

2. Test Case Design

Each user journey was implemented as an independent JUnit test class. The tests use Selenium WebDriver to interact with the actual application through a Chrome browser.

2.1 TC01 – View Inventory

Test Class: `InventoryViewTest`

The test opens the inventory page and verifies that the page loads correctly.

The following assertions are performed:

- Page title is verified.
- Inventory heading/content is verified.
- Inventory table is verified.
- Current URL is verified.

This confirms that the main inventory dashboard is accessible and contains the expected content.

2.2 TC02 – Add Inventory Item

Test Class: `AddItemTest`

The test opens the inventory page and fills the Add Item form.

The test provides:

- Item name

- Category
- Quantity
- Unit
- Price
- Reorder level

A unique item name is generated using the current system time to prevent conflicts with previously created test data.

After submitting the form, Selenium waits for the newly added item to appear in the inventory table and verifies that it is displayed.

2.3 TC03 – Search / Filter Item

Test Class: `SearchItemTest`

The test opens the inventory page and enters the search keyword `rice`.

The search form is submitted and Selenium waits for the expected item to appear in the filtered results.

The assertion verifies that the searched item is displayed.

The test uses the actual search input and search form of the application, ensuring that the automated test follows the same interaction as a normal user.

3. Transaction and Alert Testing

3.1 TC04 – Stock In and Stock Out

Test Class: `StockTransactionTest`

TC04 verifies the complete inventory transaction workflow.

The test first opens the inventory page and records the original quantity of Rice. It then opens the Transactions page and identifies the Rice item using its stable database ID.

The following operations are performed:

1. Record the original Rice quantity.
2. Select Rice in the Stock In form.
3. Add 5 units through Stock In.
4. Verify that the Stock In transaction is recorded.
5. Select the same Rice item using its stable ID.
6. Add a Stock Out transaction of 5 units.
7. Verify that the Stock Out transaction is recorded.

8. Return to the inventory page.

9. Verify that the final quantity equals the original quantity.

The test uses the database ID instead of the complete visible option text. This is important because the option text contains the current quantity, which changes after a transaction.

The final assertion verifies:

$$OriginalQuantity = FinalQuantity$$

Therefore, both Stock In and Stock Out operations are verified as part of the same user journey.

3.2 TC05 – Exception Alerts

Test Class: `ExceptionAlertsTest`

TC05 verifies the exception alert functionality.

To make the test independent of existing database data, a unique item is created with:

- Quantity = 0
- Reorder Level = 5

The test then opens the Alerts page and verifies that the newly created item appears under the Out of Stock section.

The following assertions are performed:

- Alerts page loads successfully.
- "Inventory Alerts" heading is displayed.
- The created item appears in an Out of Stock alert card.
- "OUT OF STOCK" status is displayed.
- Current quantity is displayed as zero.

4. Selenium Test Infrastructure

A reusable Selenium testing structure was created to avoid duplicating browser setup and failure-handling code in every test.

4.1 Base Selenium Test

The `BaseSeleniumTest` class provides the common Selenium configuration.

It is responsible for:

- Creating the Chrome WebDriver.
- Configuring the browser window size.

- Providing the common application base URL.

All five critical Selenium test classes extend this base class.

4.2 Explicit Waits

The tests use Selenium's `WebDriverWait` instead of relying on fixed delays.

For example:

```
WebDriverWait wait =
        new WebDriverWait(driver, Duration.ofSeconds(10));
```

Explicit waits allow Selenium to wait until a required element or application state is available.

4.3 Failure Screenshot Mechanism

A JUnit extension named `ScreenshotOnFailureExtension` was implemented.

The extension checks whether a test has failed after its execution. If a failure is detected, Selenium captures the current browser screen.

Screenshots are automatically stored inside:

```
target/screenshots/
```

The filename contains the test name and timestamp.

Example:

```
performStockInAndStockOut____20260922_154735.png
```

The screenshot mechanism was intentionally tested by introducing test failures during development. The resulting screenshots confirmed that the failure capture mechanism works correctly.

5. Local Test Execution

The complete test suite was executed locally using Maven.

The command used for test execution was:

```
./mvnw test
```

Maven Surefire was responsible for discovering and executing the JUnit tests.

The final test execution produced the following results:

Result	Value
Total Tests Executed	7
Selenium Tests	6
Existing Spring Boot Test	1
Errors	0
Failures	0
Skipped Tests	0
Success Rate	100%

Table 1: Final Local Test Execution Result

The five required critical Selenium journeys completed successfully:

- `InventoryViewTest`
- `AddItemTest`
- `SearchItemTest`
- `StockTransactionTest`
- `ExceptionAlertsTest`

An additional Selenium setup test was also executed as part of the local test suite.

6. Local Test Report

The Maven Surefire HTML report was generated using:

```
./mvnw surefire-report:report-only
```

The generated report showed:

```
Tests:      7
Errors:     0
Failures:   0
Skipped:    0
Success:    100%
```

The Selenium test package achieved a 100% success rate.

The individual Selenium test results were:

Test Class	Tests	Result
InventoryViewTest	1	PASS
AddItemTest	1	PASS
SearchItemTest	1	PASS
StockTransactionTest	1	PASS
ExceptionAlertsTest	1	PASS

Table 2: Critical Selenium Test Results

7. Failure Handling and Debugging

During development, failures were intentionally introduced to verify the screenshot mechanism.

An incorrect assertion in the inventory test successfully triggered the screenshot capture mechanism.

The Stock Transaction test also encountered a failure during development. Initially, the Rice option was selected using its complete visible text. The visible text included the current quantity, so after the Stock In operation the text changed and the subsequent Stock Out selection could not reliably find the original option text.

The test was corrected to select the item using its stable database ID.

During complete-suite execution, a `StaleElementReferenceException` was also encountered. The issue occurred because an element reference from an earlier DOM state was being reused.

The test was corrected by locating the required elements again from the current DOM before performing the Stock Out operation.

After this correction, the complete test suite executed successfully with zero failures and zero errors.

8. Conclusion

Week 9 successfully introduced automated browser-based functional testing into the Automated Grocery Inventory Dashboard.

Five critical user journeys were converted into Selenium WebDriver tests with assertions and controlled test data. The reusable Selenium base class reduced duplication, while the JUnit failure extension provided automatic screenshots for failed browser tests.

The complete test suite was executed locally through Maven and the Surefire report confirmed a 100% success rate with zero errors and zero failures.

The Week 9 implementation therefore provides a repeatable automated testing layer covering inventory viewing, item creation, search, inventory transactions and exception alerts.